Simulación

Eje Temático Nº 7

Simulación

basada en Agentes

4to Nivel
Ingeniería en Sistemas de Información

Ing. Revilla, Sergio Andrés
Ing. Sofía Gaggi

Contenidos

De dónde venimos 3
1. Qué es un agente 4
Definición 4
Qué es obligatorio y qué es opcional 5
Cuatro cosas que un agente no es 5
2. Anatomía de un modelo basado en agentes 6
Los ocho componentes 6
Cómo avanza el tiempo 7
El orden de activación 7
3. Emergencia 9
Qué es 9
Cómo reconocerla 9
Tres casos para poner a prueba el criterio 9
Una advertencia 10
4. Un modelo trazado a mano 11
El modelo de Schelling 11
La configuración 12
Cómo se evalúa un agente 13
La grilla completa 13
Un tick 14
Qué muestra el ejemplo 16
5. Cuándo conviene usar agentes y cuándo no 17
Agentes frente a eventos discretos 17
Un mismo sistema, dos miradas 17

Agentes frente a dinámica de sistemas 18
Señales a favor y señales en contra 18
6. Áreas de aplicación 19
Aplicaciones en Ingeniería en Sistemas 19
Referencias bibliográficas 20
Anexo A. Breve historia del paradigma 22
Anexo B. Herramientas 22
NetLogo 22
El vocabulario mínimo de NetLogo 23
NetLogo Web 23
FlexSim también hace agentes 23
Otras plataformas de modelado de agentes 24
Anexo C. El protocolo ODD 25

DE DÓNDE VENIMOS

En el Eje 4 vimos que en la simulación hay **paradigmas de modelado** distintos, y elegir uno es una decisión que se toma al formular el modelo conceptual, mucho antes de abrir un programa. Ahí nombramos cuatro: eventos discretos, dinámica de sistemas, simulación basada en agentes y multimétodo. Después dedicamos los Ejes 5 y 6 al primero: colas, medidas de desempeño, datos de entrada, réplicas, validación. Todo el Trabajo Práctico Integrador vive en ese paradigma.

Este eje es el segundo de los cuatro. Y conviene empezar mostrando qué es lo que el primero no hace naturalmente.

Volvamos al supermercado del Eje 5. Con eventos discretos podemos responder muy bien esta pregunta:

> ¿Cuántas cajas hacen falta para que la espera promedio no supere los cinco minutos?

Modelamos llegadas, colas y servidores, medimos Wq, L y ρ, probamos escenarios y contestamos. Es exactamente lo que venimos haciendo. Ahora cambiemos la pregunta:

> Los clientes miran las otras filas mientras esperan. Algunos se cambian si ven una más corta; otros aguantan. El que va con el carro lleno no se cambia nunca, y el que lleva tres productos abandona si la espera se estira. **¿Cómo afecta todo eso a la congestión?**

Esta segunda pregunta también se puede modelar con eventos discretos, pero cuesta, porque lo que importa ya no es el flujo del proceso sino **lo que decide cada cliente por su cuenta, mirando lo que tiene alrededor**. La lógica dejó de estar en la estructura del sistema y pasó a estar en los individuos.

Cuando eso ocurre, hay un paradigma que representa el sistema de esa manera desde el principio. De eso trata este eje.

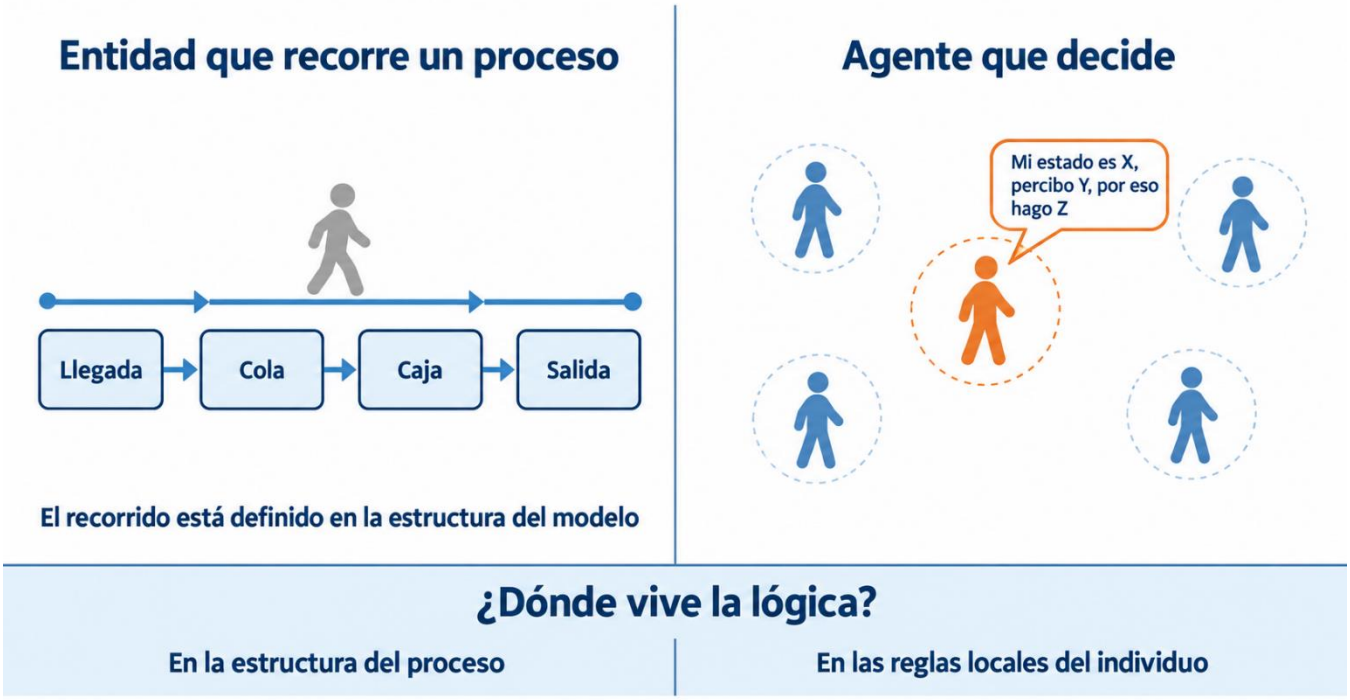
1. QUÉ ES UN AGENTE

DEFINICIÓN

> Un **agente** es una entidad individual identificable, con estado propio, situada en un entorno, que ejecuta reglas de comportamiento y puede percibir y afectar al entorno o a otros agentes con cierto grado de autonomía.

Cada parte de la definición carga algo, así que conviene desarmarla:

- **Individual e identificable.** Es un individuo, no un promedio ni un agregado. Se lo puede distinguir de los demás y conserva su identidad a lo largo de la corrida.
- **Con estado propio.** Tiene variables que le pertenecen y que lo describen en este momento. Dos agentes del mismo tipo pueden estar en estados distintos.
- **Situado en un entorno.** Existe en algún lado. Puede ser un espacio físico, una grilla, una red de contactos o cualquier otra topología. El entorno es lo que define quiénes son sus vecinos.
- **Ejecuta reglas de comportamiento.** Hace cosas, y lo que hace depende de su estado y de lo que percibe.
- **Con autonomía.** Nadie le indica paso a paso qué hacer. Corre su propia lógica.



QUÉ ES OBLIGATORIO Y QUÉ ES OPCIONAL

No todas las propiedades que suelen atribuirse a un agente son necesarias. Conviene separarlas, porque buena parte de los malentendidos vienen de exigirle a todo agente cosas que solo tienen algunos.

Propiedad	Qué significa	Estatus
Identidad	Se distingue de los demás y conserva sus atributos	**Núcleo**
Estado propio	Variables que describen su situación actual	**Núcleo**
Autonomía	Ejecuta su propia lógica sin control global	**Núcleo**
Situación en un entorno	Existe en un espacio, una red o un contexto	**Núcleo**
Reactividad	Responde a lo que percibe	Muy frecuente
Interacción	Influye sobre otros o intercambia información	Muy frecuente
Heterogeneidad	Los agentes pueden diferir entre sí	Frecuente, no obligatoria
Proactividad	Actúa orientado a objetivos propios	Opcional
Aprendizaje	Cambia sus reglas con la experiencia	Opcional
Capacidad social compleja	Negocia, coopera, comunica, sigue normas	Opcional

Las cuatro primeras son el núcleo. Todo lo demás aparece según lo que el modelo necesite, y un modelo perfectamente válido puede no tener ninguna de las otras seis.

CUATRO COSAS QUE UN AGENTE NO ES

Estos cuatro errores son los que más se repiten, y los cuatro llevan a modelar mal.

Un agente no es necesariamente una persona. Puede ser un vehículo, un router, una empresa, un animal, una célula, un nodo de una red o un proceso. Cualquier cosa que tenga estado propio y actúe según reglas locales.

Un agente no es inteligencia artificial. Los modelos más famosos del paradigma usan agentes con una sola regla de dos líneas. No hay aprendizaje, no hay razonamiento, no hay optimización. Un agente puede ser deliberadamente tonto, y eso es parte de la gracia: si un comportamiento colectivo complejo aparece a partir de reglas triviales, el hallazgo es más fuerte, no más débil.

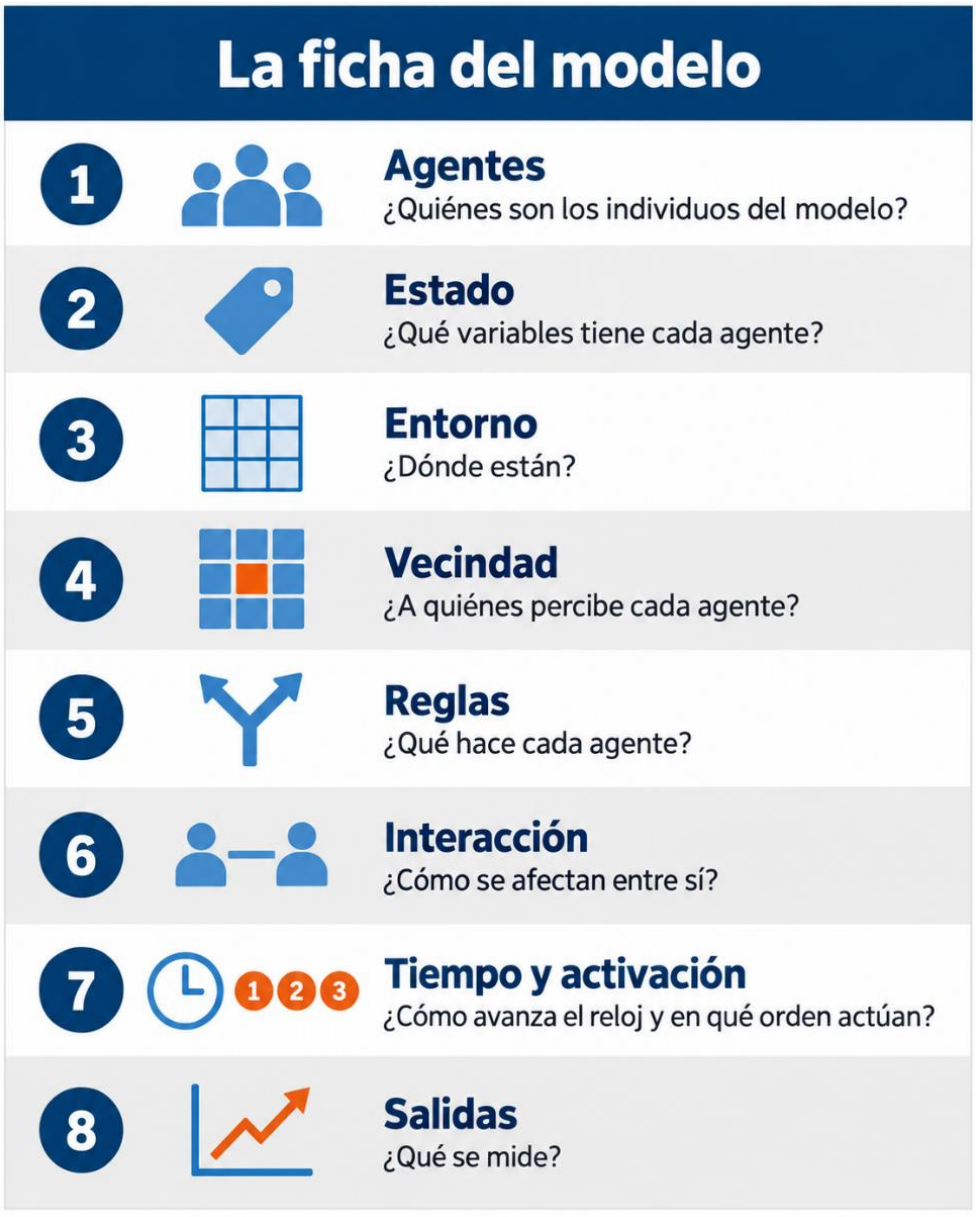
Un agente no es lo mismo que un objeto de programación. Se lo implementa como objeto, sí, pero eso es una cuestión de implementación. Lo que lo define es su papel dentro del modelo: un individuo activo que decide por sí mismo.

Un agente no es una persona animada en pantalla. Que un individuo se mueva por la pantalla no lo convierte en agente: lo que importa es si decide por sí mismo o si recorre un camino que le marca el proceso.

2. ANATOMÍA DE UN MODELO BASADO EN AGENTES

LOS OCHO COMPONENTES

En el Eje 1 vimos que antes de programar hay que formular un **modelo conceptual**: qué se incluye, qué se deja afuera, con qué supuestos. Para un modelo de agentes ese modelo conceptual tiene una estructura bastante fija. Son ocho preguntas, y si se responden las ocho, el modelo está especificado.



A esta lista la vamos a llamar **la ficha del modelo**, y la vamos a usar en el trabajo práctico: frente a cualquier modelo de agentes, completar estas ocho filas es la manera de entenderlo. Existe un protocolo estándar bastante más extenso para documentar modelos de agentes, llamado ODD; la ficha de ocho filas es una versión reducida suya, suficiente para lo que hacemos acá. En el Anexo C está el protocolo completo, para quien quiera verlo.

> En la práctica: el componente que más se olvida es el 7. Se especifican agentes, reglas y métricas, y se da por sentado que el tiempo «simplemente avanza». No es así, y las dos secciones siguientes explican por qué.

CÓMO AVANZA EL TIEMPO

En el Eje 4 estudiamos dos mecanismos de avance del tiempo: **incrementos fijos**, donde el reloj salta de a intervalos iguales, y **siguiente evento**, donde el reloj salta directamente al próximo suceso agendado. Y como en el Eje 5 usamos siempre el segundo, es fácil quedarse con esta idea:

> *Los eventos discretos usan avance al siguiente evento; los agentes usan incrementos fijos.*

Eso es falso, y conviene desarmarlo ahora. Es cierto que la mayoría de los modelos de agentes avanzan por pasos fijos —a cada paso se lo suele llamar **tick**— y que la mayoría de los modelos de eventos discretos avanzan al siguiente evento. Pero es una tendencia, no una definición. Hay modelos de agentes que agendan eventos futuros, y hay plataformas de agentes construidas sobre un motor de eventos discretos.

Lo que define a la simulación basada en agentes **no es cómo avanza el reloj, sino cómo se representa el sistema**: individuos explícitos, con estado propio, reglas locales e interacción. El mecanismo temporal es una decisión de implementación posterior.

EL ORDEN DE ACTIVACIÓN

Supongamos un modelo de contagio en el que cada agente sano que tiene un vecino enfermo se enferma. Y supongamos que en este tick A contagia a B. La pregunta es: **¿B puede contagiar a C en este mismo tick?**

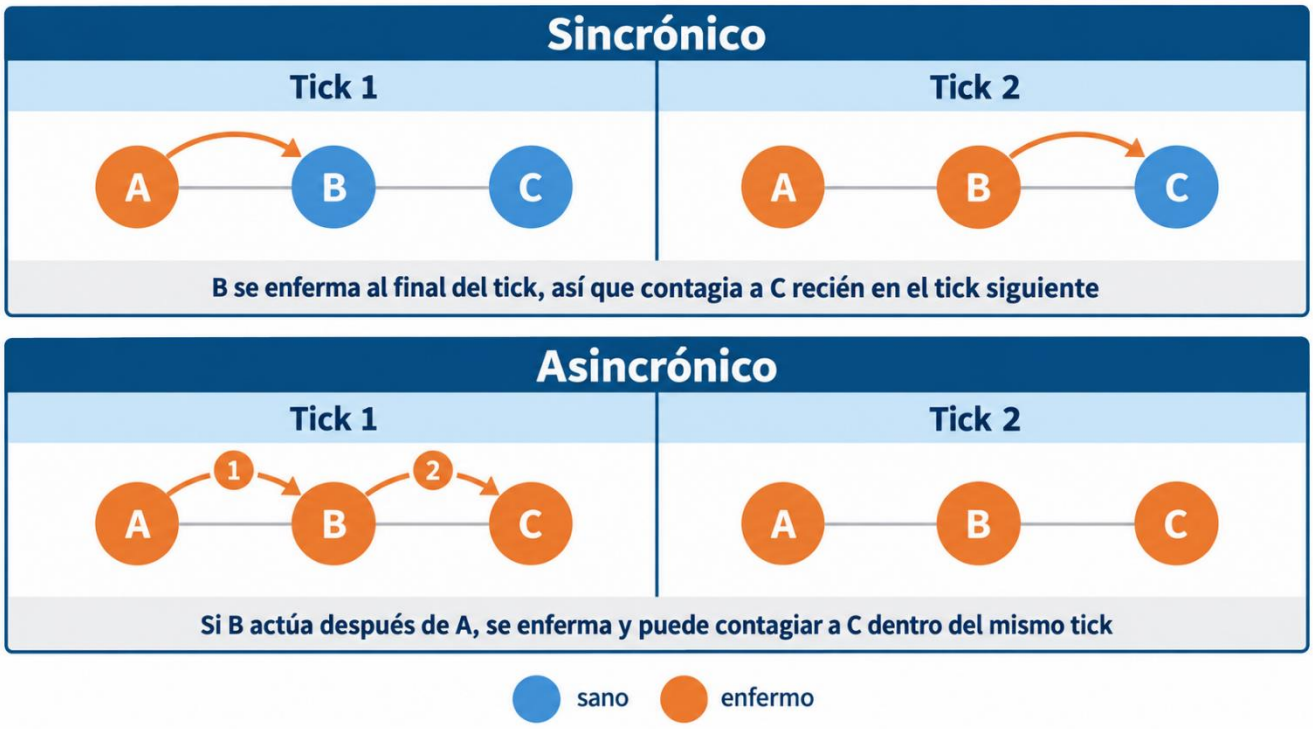
La respuesta depende de una decisión que hay que tomar explícitamente:

Esquema	Cómo funciona	Consecuencia
Sincrónico	Todos los agentes calculan su nuevo estado a partir del estado del tick anterior, y recién después se actualizan todos juntos	B se enferma al final del tick, así que contagia a C recién en el tick siguiente
Asincrónico o secuencial	Los agentes actúan de a uno, y cada uno ve los cambios que ya hicieron los anteriores	Si B actúa después de A, se enferma y puede contagiar a C dentro del mismo tick

Y si el esquema es asincrónico aparece una segunda decisión: **¿en qué orden actúan?** ¿Siempre el mismo? ¿Al azar, distinto en cada tick?

Esto no es un detalle de programación. Es parte de la especificación del modelo, y **puede cambiar los resultados**. En la sección 4 lo vamos a ver con números: el mismo estado inicial, las mismas reglas y los mismos destinos, activando a los agentes en dos órdenes distintos, dan dos resultados distintos.

> Por qué nos importa: esto conecta directamente con la **verificación** de los Ejes 4 y 6. Si dos personas implementan el mismo modelo conceptual y una actualiza sincrónicamente y la otra secuencialmente, van a obtener resultados diferentes sin que ninguna de las dos haya cometido un error de programación. La diferencia está en un supuesto que nunca se escribió. Por eso la fila 7 de la ficha del modelo no se puede dejar en blanco.



3. EMERGENCIA

QUÉ ES

> Se llama **emergencia** a la aparición de una regularidad o una estructura a nivel global que no fue programada como una regla global, sino que resulta de la interacción repetida de reglas locales.

La formulación más útil para retenerla es esta: el modelador programa **cómo actúa cada individuo**. No programa «formen una bandada», el patrón colectivo aparece solo.

Es el concepto central de esta unidad y la razón más fuerte para usar el paradigma. Cuando lo que uno quiere entender es *por qué* aparece un fenómeno colectivo, un modelo que lo produce desde abajo explica algo que un modelo que lo impone desde arriba no puede explicar.

CÓMO RECONOCERLA

Conviene tener un criterio, porque «emergente» es una palabra que se usa con demasiada facilidad. Empecemos por un test rápido:

> ¿Existe en el modelo una instrucción que ordene directamente el patrón que estoy observando?

Si la respuesta es sí, no hay emergencia: hay una regla global disfrazada. Si la respuesta es no y el patrón aparece igual, tenemos un **candidato**. Para confirmarlo hace falta el criterio completo:

> Un patrón es emergente cuando aparece a nivel global sin una regla global que lo imponga y puede explicarse como resultado acumulado de reglas e interacciones locales. La evidencia se refuerza si al modificar o eliminar una de esas reglas locales el patrón global cambia o desaparece.

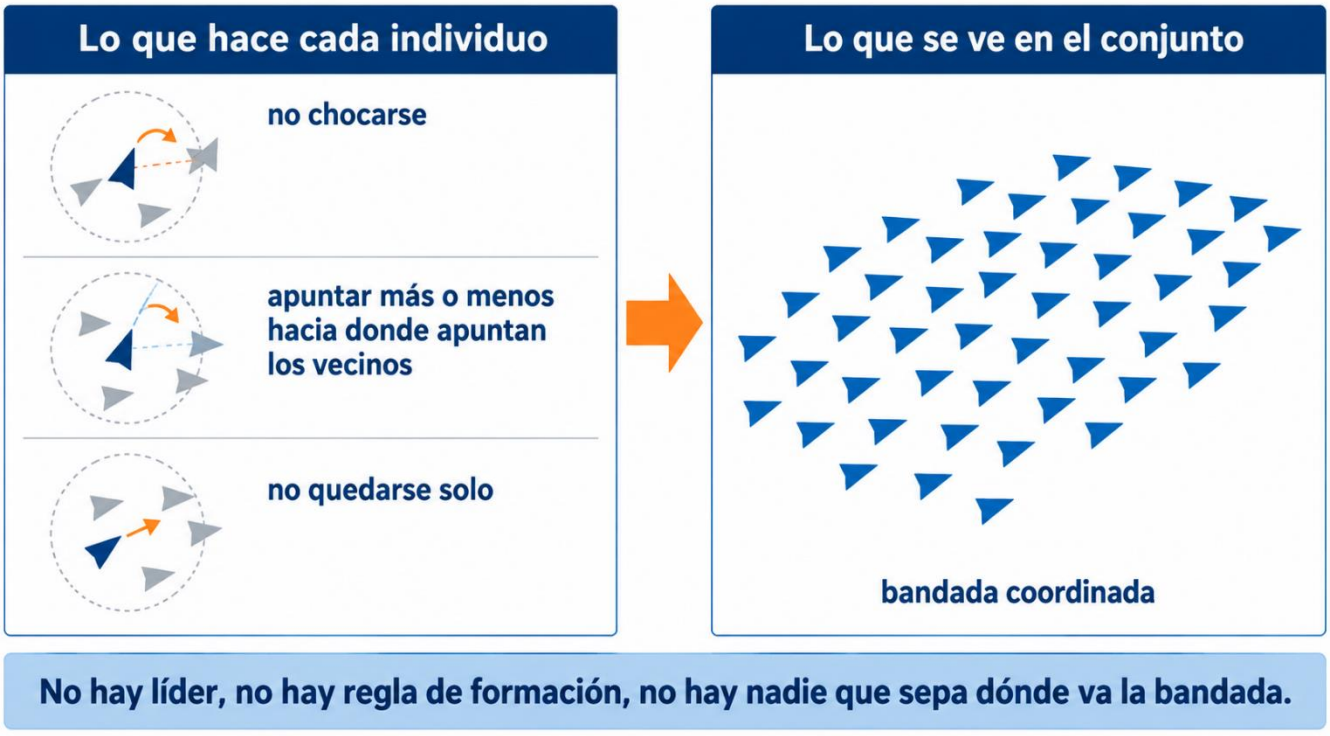
La última frase es la parte operativa, y es lo que vamos a hacer en el práctico: si sacamos una regla y el patrón se cae, tenemos evidencia de que el patrón dependía de esa regla.

TRES CASOS PARA PONER A PRUEBA EL CRITERIO

Caso 1 — No es emergencia. El modelo tiene una instrucción que, al llegar al tick 50, manda a todos los agentes de tipo A hacia la izquierda. Aparece segregación perfecta. El patrón está ahí, pero lo ordenó una regla global. El primer test ya lo descarta.

Caso 2 — Todavía no alcanza. La inicialización coloca a los agentes al azar y, por casualidad, quedan siete agentes azules juntos en un rincón. Nadie lo ordenó, así que pasa el primer test. Pero no es emergencia: no lo produjo la dinámica del modelo, lo produjo el sorteo inicial. Un patrón emergente tiene que **aparecer con las corridas**, no venir dado.

Caso 3 — Sí es emergencia. Cada pájaro aplica tres reglas mirando solo a sus vecinos cercanos: no chocarse, apuntar más o menos hacia donde apuntan ellos, y no quedarse solo. Aparece una bandada que se mueve coordinada, gira y se reagrupa. No hay líder, no hay regla de formación, no hay nadie que sepa dónde va la bandada. Pasa el primer test, y si desactivamos la regla de alineación la bandada se deshace, con lo cual también cumple el criterio completo.



UNA ADVERTENCIA

La emergencia es el concepto más atractivo del paradigma y por eso se exagera de dos maneras.

La primera exageración es tratarla como algo **misterioso o imposible de explicar**. No lo es. En modelos simples se puede analizar perfectamente por qué aparece el patrón; que no esté escrito como regla no significa que sea inexplicable.

La segunda es convertirla en **requisito**: creer que si un modelo no exhibe emergencia entonces no es un modelo de agentes. No hay acuerdo en la literatura sobre eso, y hay especialistas que sostienen expresamente lo contrario. Un modelo puede estar correctamente construido con agentes y no producir ningún patrón sorprendente. La emergencia es una razón poderosa para usar el paradigma, no un carnet de pertenencia.

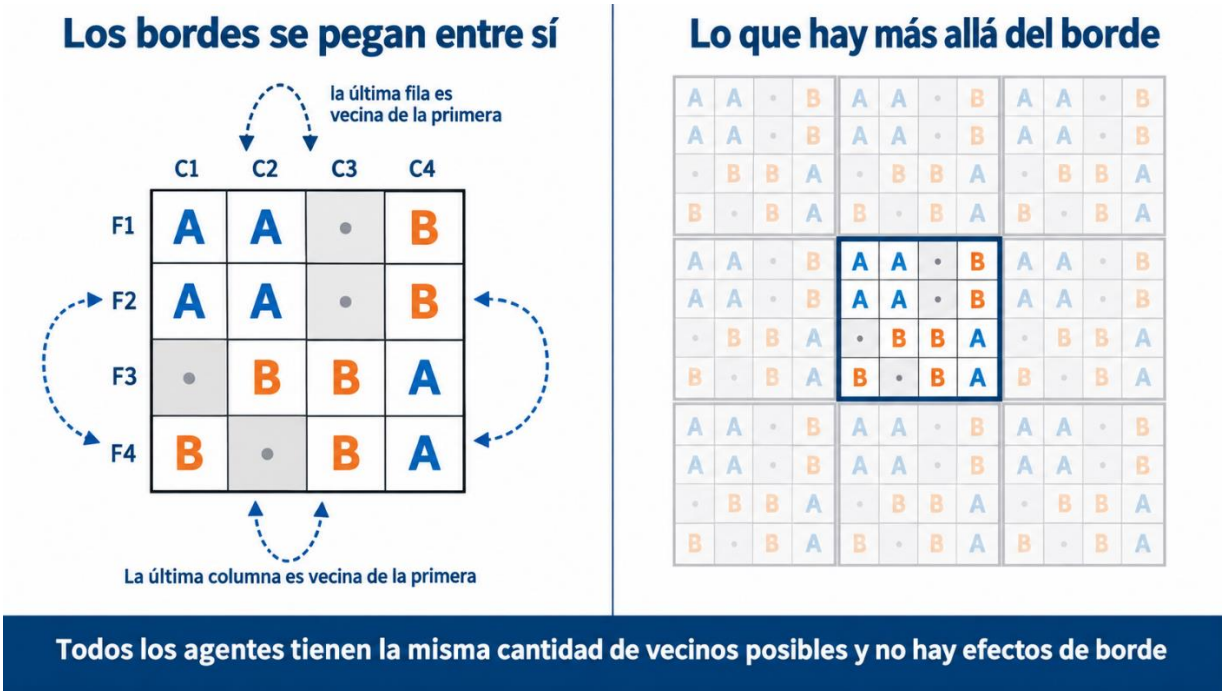
4. UN MODELO TRAZADO A MANO

Todo lo anterior se entiende mucho mejor con un modelo chico resuelto a mano, sin computadora. Vamos a usar el modelo de segregación de Schelling, que es el más conocido del paradigma y el que vas a ejecutar en el práctico.

EL MODELO DE SCHELLING

Thomas Schelling planteó en 1971 una pregunta de ciencias sociales: ¿hace falta que la gente sea intolerante para que una ciudad termine segregada? Su modelo es este:

- Hay una grilla de celdas. En cada celda puede haber **como máximo un agente**, o ninguno.
- Hay **dos tipos** de agentes, A y B. No hay ninguna asimetría entre ellos.
- La grilla es **toroidal**: los bordes se pegan entre sí. La última columna es vecina de la primera y la última fila es vecina de la primera. Así todos los agentes tienen la misma cantidad de vecinos posibles y no hay efectos de borde.
- La **vecindad** de un agente son las **ocho celdas** que lo rodean.
- Cada agente tiene un **umbral de tolerancia**: el porcentaje mínimo de vecinos del mismo tipo que necesita para estar conforme. Se cuenta solo sobre las celdas vecinas **ocupadas**; las vacías no cuentan.
- Un agente está **satisfecho** si la proporción de vecinos iguales alcanza o supera su umbral. Si no, está **insatisfecho** y se muda a una celda vacía.



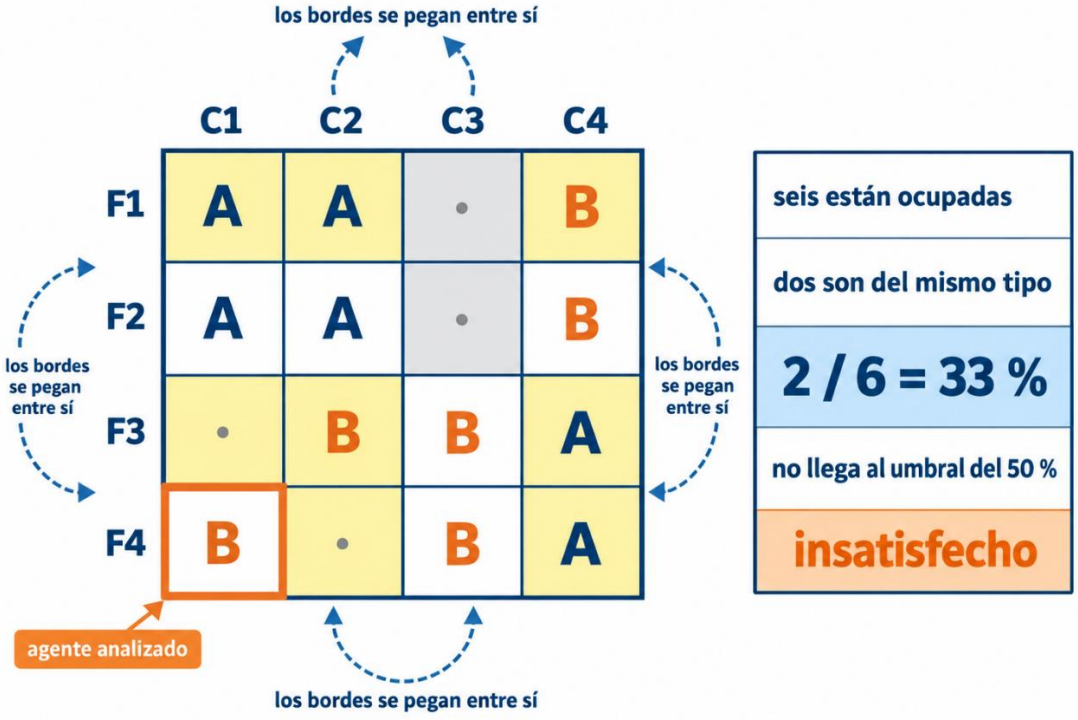
Vamos a usar un umbral del **50 %**. Es importante entender qué significa: un agente con umbral 50 % **no exige vivir solo entre iguales**. Se conforma con que la mitad de sus vecinos lo sean, y aceptaría vivir en un barrio perfectamente mezclado.

Las dos métricas globales que vamos a mirar son:

- **% de similitud:** sobre todas las relaciones de vecindad entre agentes, qué porcentaje une a dos agentes del mismo tipo. Mide qué tan segregada está la grilla.
- **Cantidad de insatisfechos:** cuántos agentes no cumplen su umbral.

LA CONFIGURACIÓN

Una grilla de 4 × 4 con 12 agentes, seis de cada tipo, y cuatro celdas vacías. El punto marca una celda vacía.



Estado inicial	C1	C2	C3	C4
F1	A	A	·	B
F2	A	A	·	B
F3	·	B	B	A
F4	B	·	B	A

CÓMO SE EVALÚA UN AGENTE

Tomemos el agente **B de la fila 4, columna 1**, y contemos sus vecinos. La grilla es toroidal, así que a su izquierda está la columna 4 y debajo suyo está la fila 1. Sus ocho vecinas son F3C4, F3C1, F3C2, F4C4, F4C2, F1C4, F1C1 y F1C2. Veamos qué hay en cada una:

Celda vecina	F3C4	F3C1	F3C2	F4C4	F4C2	F1C4	F1C1	F1C2
Contenido	A	vacía	B	A	vacía	B	A	A

De las ocho celdas, **seis están ocupadas**. De esas seis, **dos** tienen un agente del mismo tipo. La proporción de iguales es 2/6 = 33 %, que no llega al umbral del 50 %. Este agente está **insatisfecho** y se va a mudar.

> Notá que las dos celdas vacías no cuentan ni a favor ni en contra. El cálculo se hace únicamente sobre los vecinos que existen. Si el agente tuviera siete vecinas vacías y una sola ocupada por un agente igual, estaría satisfecho con el 100 %.

LA GRILLA COMPLETA

Repitiendo la cuenta para los doce agentes:

Agente	Posición	Iguales / ocupados	Proporción	Estado
A	F1C1	4 / 7	57 %	Satisfecho
A	F1C2	3 / 5	60 %	Satisfecho
B	F1C4	3 / 6	50 %	Satisfecho
A	F2C1	4 / 7	57 %	Satisfecho
A	F2C2	3 / 5	60 %	Satisfecho
B	F2C4	2 / 5	40 %	**Insatisfecho**
B	F3C2	3 / 5	60 %	Satisfecho
B	F3C3	3 / 6	50 %	Satisfecho
A	F3C4	2 / 6	33 %	**Insatisfecho**
B	F4C1	2 / 6	33 %	**Insatisfecho**
B	F4C3	3 / 6	50 %	Satisfecho
A	F4C4	2 / 6	33 %	**Insatisfecho**
Total		**34 / 70**	**48,57 %**	**4 insatisfechos**

Notá que cuatro agentes quedan **exactamente en el 50 %** y están satisfechos: la regla pide **alcanzar** el umbral, no superarlo.

Los indicadores globales salen directamente de esta tabla, sin ningún cálculo nuevo. El total de vecinos ocupados es **la suma de la columna de ocupados**, y el total de vecinos del mismo tipo es **la suma de la columna de iguales**. Cada vecindad queda contada dos veces, una desde cada uno de los dos agentes que la forman, pero eso no cambia el porcentaje.

Indicador	Valor
Vecinos ocupados, sumando los de los doce agentes	70
De ellos, del mismo tipo	34
% de similitud	**34 / 70 = 48,57 %**
Agentes insatisfechos	**4** de 12

UN TICK

Ahora ejecutamos un paso. Los cuatro agentes insatisfechos se mudan a las cuatro celdas vacías. Para poder hacerlo a mano fijamos de antemano el orden en que se van a ocupar los destinos:

> F1C3 → F2C3 → F3C1 → F4C2

El primer agente que se activa toma F1C3, el segundo toma F2C3, y así. Falta una sola cosa: **decidir en qué orden se activan los cuatro agentes**. Y acá es donde aparece lo interesante.

Orden 1. Activamos así: B(F2C4) → A(F3C4) → B(F4C1) → A(F4C4).

Orden 1	C1	C2	C3	C4
F1	A	A	B	B
F2	A	A	A	·
F3	B	B	B	·
F4	·	A	B	·

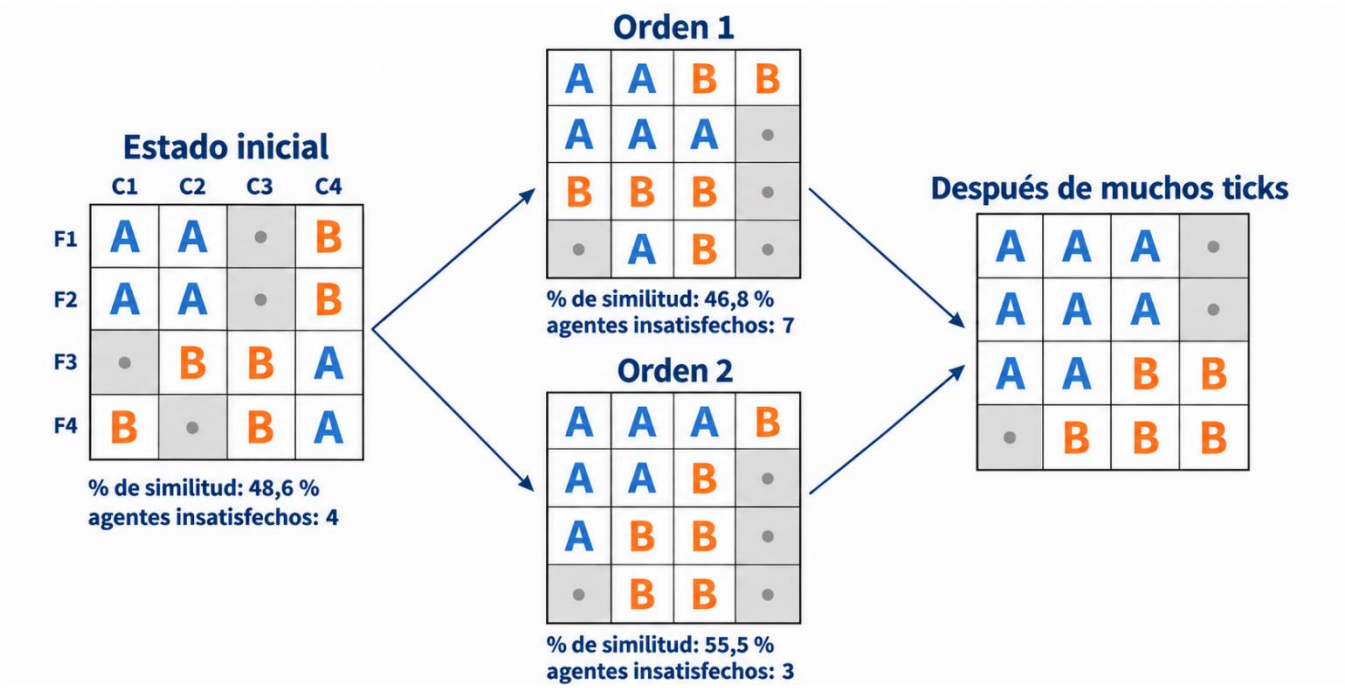
Indicador	Valor
% de similitud	**47,22 %**
Agentes insatisfechos	**7**

Orden 2. Mismo estado inicial, mismas reglas, misma secuencia de destinos. Lo único que cambia es el orden de activación, que ahora es el inverso: A(F4C4) → B(F4C1) → A(F3C4) → B(F2C4).

Estado inicial	C1	C2	C3	C4
F1	A	A	·	B
F2	A	A	·	B
F3	·	B	B	A
F4	B	·	B	A

Orden 2	C1	C2	C3	C4
F1	A	A	A	B
F2	A	A	B	·
F3	A	B	B	·
F4	·	B	B	·

Indicador	Valor
% de similitud	**55,56 %**
Agentes insatisfechos	**3**



QUÉ MUESTRA EL EJEMPLO

Primero: el orden de activación cambia el resultado. Partiendo del mismo estado, con las mismas reglas y los mismos destinos, un orden deja la grilla con 47,22 % de similitud y siete insatisfechos, y el otro con 55,56 % y tres. Ninguno de los dos está mal. Son dos modelos distintos, porque el orden de activación **es parte de la especificación**. Si esto no se declara, dos implementaciones del mismo modelo conceptual van a dar resultados distintos y nadie va a saber por qué.

Segundo: el sistema no mejora prolijamente tick a tick. En el Orden 1 la cantidad de insatisfechos *subió* de 4 a 7. No es un error: cuando un agente se muda deja huérfanos a los vecinos que se quedaron, y además puede caer en un lugar donde tampoco está cómodo. Un modelo de agentes no es un algoritmo de optimización que avanza hacia una solución; es un sistema que evoluciona, y hay que dejarlo correr muchos ticks para ver a dónde va.

Tercero, y es el punto central: nadie programó la segregación. Volvé a leer las reglas. No hay ninguna instrucción que diga «formá barrios», ni «juntate con los de tu tipo», ni «alejate de los otros». Lo único que hay es un agente que se muda si menos de la mitad de sus vecinos son como él, es decir, un agente que **aceptaría vivir en un barrio mitad y mitad**. Y sin embargo, cuando el modelo corre muchos ticks, la grilla se segrega.

Ese resultado es lo que hizo famoso al modelo de Schelling, y es exactamente lo que definimos en la sección 3: un patrón global que aparece sin que ninguna regla global lo imponga. Aplicale el primer test —¿hay una instrucción que ordene formar barrios?— y vas a ver que no la hay.

> Y una lectura que va más allá del modelo: de acá no se sigue que la segregación real se explique así. El modelo muestra algo más acotado y más incómodo, que **una preferencia individual moderada alcanza para producir un resultado colectivo extremo**. Que el resultado global sea segregado no permite concluir que los individuos lo hayan querido. Esa distinción entre lo que quiere cada uno y lo que produce el conjunto es, en el fondo, de qué se trata este paradigma.

5. CUÁNDO CONVIENE USAR AGENTES Y CUÁNDO NO

AGENTES FRENTE A EVENTOS DISCRETOS

La comparación con lo que ya sabemos es la mejor manera de ubicar el paradigma. Pero hay que hacerla con cuidado, porque la frontera no es tan nítida como suele contarse.

Aspecto	Eventos discretos	Agentes
Unidad conceptual	Proceso, evento, recurso, cola	Agente, estado, regla, entorno
La entidad típica	Recorre un flujo definido en el modelo	Ejecuta su propio comportamiento
Dónde está la lógica	En la estructura del proceso	En las reglas locales del individuo
Heterogeneidad	Posible	Frecuentemente central
Vecindad o red	Posible, pero no es la abstracción dominante	Natural y explícita
Pregunta típica	Espera, utilización, capacidad	Difusión, interacción, adaptación, micro → macro

> Advertencia: esta tabla es una guía de modelado, no una frontera informática. Una entidad de un modelo de eventos discretos puede llevar atributos, rutearse según condiciones y disparar eventos. Un agente puede tener una regla trivial. La diferencia más defendible es **dónde se ubica la lógica**: en el flujo del sistema o en la perspectiva del individuo que dice «mi estado es X, percibo Y, por eso hago Z».

UN MISMO SISTEMA, DOS MIRADAS

El supermercado del Eje 5 admite las dos lecturas, y ninguna es más correcta que la otra: son respuestas a preguntas distintas.

Sistema	Con eventos discretos	Con agentes
Supermercado	Llegada → cola → caja → salida. Sirve para dimensionar cajas y estimar esperas.	Cada cliente elige caja, mira las otras filas, se cambia o abandona según su propia tolerancia.
Guardia de hospital	Flujo por triage, consultorio, laboratorio y camas. Sirve para capacidad.	Contagio entre personas, movimiento por el edificio, cumplimiento individual de protocolos.
Red de datos	Paquetes que atraviesan colas en los nodos. Sirve para latencia y rendimiento.	Nodos que deciden su ruteo, se congestionan, fallan y se adaptan a lo que ven sus vecinos.

Notá el patrón: la columna del medio pregunta **cuánto**, y la de la derecha pregunta **qué pasa si los individuos se comportan distinto entre sí**.

AGENTES FRENTE A DINÁMICA DE SISTEMAS

El Eje 8 trata el tercer paradigma, la simulación continua o dinámica de sistemas, así que vale la pena dejar planteado el contraste desde acá.

Dinámica de sistemas	Agentes
Trabaja con niveles agregados y flujos	Trabaja con individuos explícitos
Ecuaciones y bucles de realimentación	Reglas de comportamiento e interacciones
Observa la trayectoria del agregado	Observa el individuo y el agregado a la vez
Muy eficiente cuando los promedios alcanzan	Más costoso, porque representa a cada individuo

El ejemplo clásico es una epidemia. Con dinámica de sistemas se representan tres niveles —susceptibles, infectados y recuperados— y las tasas promedio de pasaje entre ellos. Con agentes, cada persona conserva su edad, su hogar, su red de contactos y su movilidad, y la curva agregada aparece como resultado de los contagios individuales. Ninguno de los dos es «más realista» por definición: si la pregunta se responde con promedios, el modelo de dinámicas de sistemas es mejor porque es más barato y tiene menos supuestos.

SEÑALES A FAVOR Y SEÑALES EN CONTRA

Los agentes se justifican especialmente cuando aparece alguna de estas señales:

- Importa la **heterogeneidad**: los individuos no son intercambiables.
- Importa **quién interactúa con quién**: hay redes, vecindad o espacio.
- Los actores toman **decisiones locales** con información parcial.
- Hay **adaptación o aprendizaje**.
- Se quiere **explicar** un patrón macro a partir de reglas micro.
- Una política actúa **de manera selectiva** sobre subgrupos distintos.

Y conviene descartarlos cuando ocurre alguna de estas otras:

- La pregunta es de capacidad, cola o secuencia, y los eventos discretos la resuelven de manera natural.
- Un modelo de dinámica de sistemas responde lo mismo con menos supuestos.
- No hay evidencia para justificar las reglas individuales que habría que inventar.
- Se representan miles de individuos solo para terminar mirando un promedio.
- El detalle micro no cambia la decisión que hay que tomar.

Advertencia sobre el costo oculto: un modelo de agentes obliga a inventar reglas de comportamiento, y esas reglas son mucho más difíciles de medir que un tiempo de servicio. Un modelo puede reproducir perfectamente los datos con reglas equivocadas; es lo que se llama equifinalidad, y es la crítica metodológica más seria al paradigma.

6. ÁREAS DE APLICACIÓN

Lo que hace apropiado al paradigma no es el dominio en sí, sino que la pregunta dependa de la interacción y de las diferencias entre individuos.

Dominio	Pregunta típica	Qué aporta el enfoque
Epidemiología	¿Cómo cambia el contagio si los hogares, las escuelas y las edades difieren?	Contactos heterogéneos y redes explícitas
Tránsito y movilidad	¿Cómo producen congestión las decisiones individuales de ruta y horario?	Vehículos y viajeros representados uno por uno
Evacuación y multitudes	¿Qué salidas eligen las personas y cómo se estorban entre sí?	Percepción del entorno y fuerzas de interacción
Mercados y economía	¿Qué patrones resultan de estrategias distintas entre participantes?	Compradores y vendedores con reglas propias
Difusión de innovaciones	¿Cómo se propaga la adopción de un producto por una red social?	Influencia local y umbrales individuales
Ecología	¿Cómo interactúan individuos y especies con el espacio y los recursos?	Heterogeneidad y espacio explícitos
Cadena de suministro	¿Cómo responden empresas autónomas ante una perturbación?	Decisiones distribuidas sin coordinación central
Urbanismo	¿Cómo producen los hogares patrones de uso del suelo?	Elección individual sobre un espacio

APLICACIONES EN INGENIERÍA EN SISTEMAS

Cuatro que quedan cerca de lo que estudiamos en la carrera:

- **Redes de computadoras.** Cada nodo, router o host es un agente con estado local, política de ruteo y posibilidad de fallar. Es especialmente útil cuando los nodos adaptan su conducta y lo que interesa es la resiliencia del conjunto.

- **Sistemas distribuidos.** Procesos, servicios o dispositivos IoT que cooperan, alcanzan consenso, reparten carga o propagan una falla. La lógica está por definición en cada nodo, así que el paradigma calza naturalmente.
- **Ciberseguridad.** Atacantes, usuarios, hosts y defensas como agentes con estrategias distintas. Permite estudiar propagación de malware, campañas de phishing y respuesta distribuida.
- **Usuarios de una plataforma.** Usuarios heterogéneos que adoptan, abandonan, comparten o reaccionan al contenido. Sirve para estudiar efectos de red, cascadas de difusión y el impacto de una política de moderación.

REFERENCIAS BIBLIOGRÁFICAS

Bibliografía obligatoria

- Apunte de la Materia.

Complementaria en español

- AGUILERA ONTIVEROS, Antonio; POSADA CALVO, Marta. *Introducción al modelado basado en agentes. Una aproximación desde NetLogo*. El Colegio de San Luis, 2017. Disponible en formato digital abierto.
- IZQUIERDO, Luis R.; GALÁN, José M.; SANTOS, José I.; DEL OLMO, Ricardo. «Modelado de sistemas complejos mediante simulación basada en agentes y mediante dinámica de sistemas». *EMPIRIA*, Nº 16, 2008, pp. 85-112. Especialmente útil para el contraste con el Eje 8.
- GINOVART, Marta y otros. «Modelos basados en el individuo y la plataforma NetLogo». *UNIÓN — Revista Iberoamericana de Educación Matemática*, Nº 27, 2011, pp. 131-150.

Complementaria en inglés

- WILENSKY, Uri; RAND, William. *An Introduction to Agent-Based Modeling: Modeling Natural, Social, and Engineered Complex Systems with NetLogo*. MIT Press, 2015.
- RAILSBACK, Steven F.; GRIMM, Volker. *Agent-Based and Individual-Based Modeling: A Practical Introduction*. 2.ª ed. Princeton University Press, 2019.
- LAW, Averill M. *Simulation Modeling and Analysis*. 6.ª ed. McGraw-Hill, 2024.
- LAW, Averill M. «Discrete-Event and Agent-Based Simulation and Where to Use Each». *Proceedings of the 2015 Winter Simulation Conference*.
- MACAL, Charles M.; NORTH, Michael J. «Tutorial on Agent-Based Modelling and Simulation». *Journal of Simulation*, 4, 2010, pp. 151-162.
- GRIMM, Volker y otros. «Pattern-Oriented Modeling of Agent-Based Complex Systems: Lessons from Ecology». *Science*, 310(5750), 2005, pp. 987-991.

- GRIMM, Volker y otros. «The ODD Protocol for Describing Agent-Based and Other Simulation Models: A Second Update». *JASSS*, 23(2), 2020.
- SCHELLING, Thomas C. «Dynamic Models of Segregation». *Journal of Mathematical Sociology*, 1(2), 1971, pp. 143-186.
- REYNOLDS, Craig W. «Flocks, Herds and Schools: A Distributed Behavioral Model». *Proceedings of SIGGRAPH*, 1987.
- WINDRUM, Paul; FAGIOLO, Giorgio; MONETA, Alessio. «Empirical Validation of Agent-Based Models: Alternatives and Prospects». *JASSS*, 10(2), 2007.

Software y documentación

- NetLogo, Center for Connected Learning and Computer-Based Modeling, Northwestern University. Biblioteca de modelos y manual oficial.
- NetLogo Web — versión en navegador.
- FlexSim, documentación de Agent System y Object Process Flow.

Atribución: el modelo Segregation utilizado en este apunte y en el trabajo práctico pertenece a la biblioteca de modelos de NetLogo, es obra de Uri Wilensky y del Center for Connected Learning de la Universidad Northwestern, y se distribuye bajo licencia Creative Commons Atribución-NoComercial-CompartirIgual.

ANEXO A. BREVE HISTORIA DEL PARADIGMA

El paradigma no apareció de golpe. Estas son las estaciones principales:

Año	Hito
1940-50	Von Neumann y los autómatas celulares: la idea de que celdas con reglas locales pueden producir comportamiento complejo.
1970	El _Juego de la Vida_ de Conway populariza esa idea fuera del ámbito académico.
1971	Schelling publica sus modelos de segregación, trabajados originalmente a mano sobre un tablero.
1987	Reynolds publica los _boids_: tres reglas locales que producen bandadas, con enorme impacto posterior en gráficos por computadora.
1996	Epstein y Axtell publican _Growing Artificial Societies_ y consolidan el enfoque de construir sociedades artificiales desde abajo.
1999	Wilensky desarrolla NetLogo, que vuelve accesible el paradigma a estudiantes sin formación en programación avanzada.
2000-10	Se consolidan Repast, MASON y AnyLogic, y el tema entra de forma estable en la Winter Simulation Conference.
2024	La 6.ª edición de Law incorpora tratamiento dedicado de simulación basada en agentes dentro de un tratado general de simulación.

Vale la pena notar el detalle de 1971: Schelling trabajó su modelo **sin computadora**, moviendo fichas sobre un tablero. Es exactamente lo que hicimos en la sección 4, y es una buena señal de que el paradigma se puede entender sin software.

ANEXO B. HERRAMIENTAS

En el Eje 4 vimos que cada paradigma tiene su propio software y que la elección del paradigma es previa a la elección de la herramienta. Acá se cumple exactamente eso.

NETLOGO

NetLogo es el entorno de modelado basado en agentes más usado en enseñanza. Fue creado por Uri Wilensky en 1999 y se sigue desarrollando en la Universidad Northwestern. Es libre y gratuito. Su ventaja no es la potencia sino el diseño: empezar es muy fácil, y sin embargo permite construir modelos serios.

Tres cosas lo hacen especialmente apto para nosotros:

- Trae una **biblioteca extensa de modelos** ya construidos, documentados y listos para ejecutar, entre ellos los clásicos del paradigma.

- El lenguaje es **chico**: se puede leer y modificar un modelo sin haberlo estudiado.
- Tiene una **versión que corre en el navegador**, sin instalar nada.

EL VOCABULARIO MÍNIMO DE NETLOGO

No vamos a aprender el lenguaje. Alcanza con cinco palabras para poder leer un modelo:

Término	Qué es
turtle	Un agente móvil. Es el agente en el sentido de este eje.
patch	Una celda fija del terreno. También tiene estado propio y puede ejecutar reglas.
link	Una conexión entre dos agentes. Es lo que permite modelar redes.
observer	El nivel global: crea los agentes, les da órdenes y mide los indicadores.
tick	Un paso de tiempo del modelo.

Con eso ya se puede entender la estructura de casi cualquier modelo de la biblioteca: hay un botón que inicializa, un botón que ejecuta ticks, unos deslizadores que fijan los parámetros y unos gráficos que muestran las métricas globales.

NETLOGO WEB

NetLogo Web es la versión que corre en el navegador. No requiere instalación ni permisos, y ejecuta sin problemas los modelos que vamos a usar. Tiene menos funciones que la versión de escritorio —por ejemplo, no incluye la herramienta de experimentación por lotes— pero para el alcance de esta unidad alcanza y sobra.

> Importante para el práctico: si modificás un modelo en NetLogo Web y cerrás la pestaña, se pierde. Para conservarlo hay que exportarlo desde el menú, que ofrece dos formatos: un archivo de modelo, que se vuelve a abrir en NetLogo, y un archivo HTML autocontenido que ejecuta el modelo en cualquier navegador sin conexión.

FLEXSIM TAMBIÉN HACE AGENTES

Podría parecer que FlexSim, la herramienta del Trabajo Práctico Integrador, es «de eventos discretos» y nada más. No es así. Desde 2020 incluye una herramienta llamada **Agent System** que permite construir comportamientos basados en agentes: agentes con reglas propias y heterogéneas, detección de proximidad, consulta de vecinos, comportamientos que se activan y desactivan, y fuerzas sociales para el movimiento de multitudes. Además permite definir la lógica **desde la perspectiva del objeto**, que es precisamente la forma de pensar de este paradigma.

La propia documentación de FlexSim aclara que su sistema de agentes se relaciona estrechamente, aunque no exactamente, con lo que la literatura llama modelado basado en agentes. Es una implementación concreta, con sus propias abstracciones, no la definición general del paradigma.

> Por qué nos sirve saberlo: primero, porque desarma la idea de que cada software pertenece para siempre a un solo paradigma. Segundo, porque nos devuelve a una de las cuatro cosas que un agente no es, en la sección 1: en FlexSim se puede hacer simulación basada en agentes, pero **no se hace sola por poner personas que caminan**. Hay que construir el comportamiento desde el individuo.

Si en NetLogo lo natural es un modelo abstracto de reglas sobre una grilla, en FlexSim lo natural es lo contrario: un espacio físico realista con individuos que se detectan por proximidad. Un modelo de contagio entre personas que se cruzan en un pasillo le sale mucho más cómodo a FlexSim que a NetLogo, y la segregación de Schelling, al revés.

OTRAS PLATAFORMAS DE MODELADO DE AGENTES

Plataforma	Licencia	Lenguaje	Fortaleza	Dificultad inicial
NetLogo	Libre (GPL)	NetLogo	Biblioteca de modelos y versión web; diseñado para enseñar	Muy baja
AnyLogic	Comercial, con edición académica limitada	Java más interfaz gráfica	Multimétodo real: los tres paradigmas en un mismo modelo	Media
Mesa	Libre (Apache 2.0)	Python	Integración con el ecosistema de datos de Python	Baja
Repast Simphony	Libre	Java, Groovy, ReLogo	Modelos grandes, redes, datos geográficos	Media-alta
GAMA	Libre	GAML	Modelos espacialmente explícitos y datos geográficos	Media
MASON	Libre	Java	Eficiencia y experimentos masivos	Alta
FlexSim	Comercial	FlexScript más interfaz gráfica	Entorno industrial 3D; combina agentes con eventos discretos	Baja si ya se conoce

Para un primer contacto, la combinación de biblioteca de modelos, lenguaje chico y versión en navegador hace que NetLogo sea difícil de superar. Para un trabajo profesional posterior, la elección depende de si lo que pesa es la integración con datos, el espacio geográfico, la escala o la convivencia con modelos de eventos discretos.

ANEXO C. EL PROTOCOLO ODD

La ficha de ocho filas de la sección 2 es una versión reducida de un protocolo estándar llamado **ODD**, por *Overview, Design concepts, Details*. Nació en ecología, ante un problema concreto: los modelos de agentes publicados se describían de manera tan incompleta que era imposible reproducirlos. Hoy es la referencia habitual para documentarlos, y su versión vigente es una actualización de 2020.

El protocolo organiza la descripción en siete elementos, agrupados en tres bloques:

Bloque	Elemento	Qué contiene
Overview	Propósito	Para qué se construyó el modelo y qué pregunta responde
Overview	Entidades, variables de estado y escalas	Qué tipos de agente hay, qué variables tienen, qué representa un tick y una celda
Overview	Visión general del proceso y programación	Qué ocurre en cada paso y en qué orden actúan los agentes
Design concepts	Conceptos de diseño	Emergencia, adaptación, objetivos, aprendizaje, percepción, interacción, estocasticidad, observación
Details	Inicialización	Con qué estado arranca el modelo
Details	Datos de entrada	Qué información externa consume durante la corrida
Details	Submodelos	El detalle de cada regla de comportamiento

> Advertencia: ODD documenta, no valida. Un modelo puede estar impecablemente descripto según el protocolo y ser un mal modelo. Sirve para que otro pueda reproducirlo y discutirlo, que ya es bastante.

Notá que los tres elementos del bloque *Overview* son, esencialmente, las ocho filas de nuestra ficha. La diferencia está en el bloque de conceptos de diseño, que obliga a declarar explícitamente cuestiones que de otro modo quedan implícitas.